

# Proyecto 1 - IE0435

Guillermo Escobar Arrieta

April 2026

## 1 Introducción

En este documento se detalla el procedimiento usado para entrenar distintos modelos de IA capaces de clasificar contaminaciones en una línea de producción simulada. El código completo de generación del conjunto de datos y entrenamiento se encuentra en el repositorio de Github: [https://github.com/guesar2/proyecto1\\_ie0435.git](https://github.com/guesar2/proyecto1_ie0435.git). En el README del repositorio se encuentran los detalles de la ejecución del código.

## 2 Preprocesamiento y conformación del conjunto de datos

Las imágenes con las fotografías tomadas se convirtieron a imágenes en blanco y negro de tamaño  $128 \times 128$  usando la biblioteca OpenCV en el lenguaje de programación Python. Con la misma biblioteca se transformó la imagen en una matriz binaria con un valor de 1 para píxeles en blanco y 0 para píxeles con presencia de objetos. La matriz se convirtió en un vector fila para cada imagen. Estos vectores fila se almacenaron en un archivo CSV que contiene en cada fila los bits de una imagen.

Aunque esta representación simplifica significativamente el problema y reduce el costo computacional del entrenamiento, también implica la pérdida de información relacionada con intensidad, textura y gradientes de color que podría ser útil para la clasificación. Sin embargo, debido a la naturaleza relativamente simple de las imágenes utilizadas y al objetivo del proyecto, esta representación se consideró adecuada para una primera aproximación al problema.

## 3 Entrenamiento

El conjunto de datos previamente construido, junto con los conjuntos de datos construidos por otros dos compañeros de clase, se utilizó para entrenar cuatro algoritmos clásicos de clasificación: árboles de decisión, Naive Bayes, k-nearest neighbors (KNN) y máquinas de soporte vectorial (SVM).

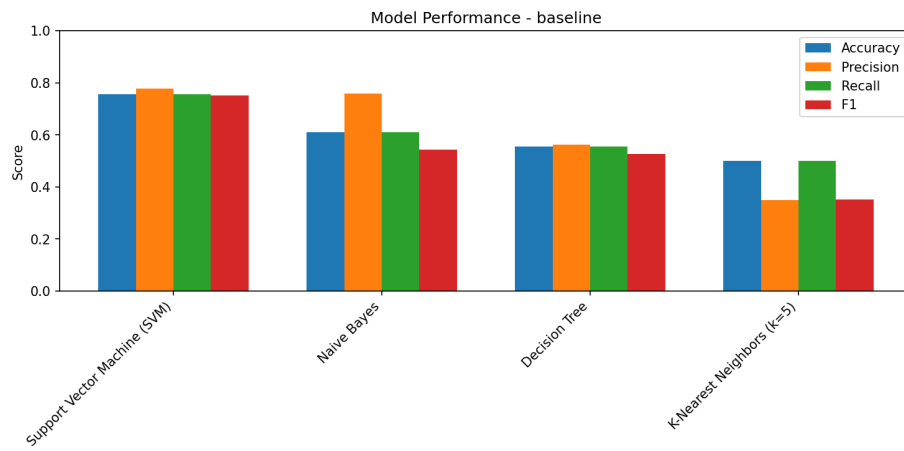


Figure 1: Comparación de las métricas evaluadas sobre todos los modelos. Cada una se calculó usando funciones de scikit-learn.

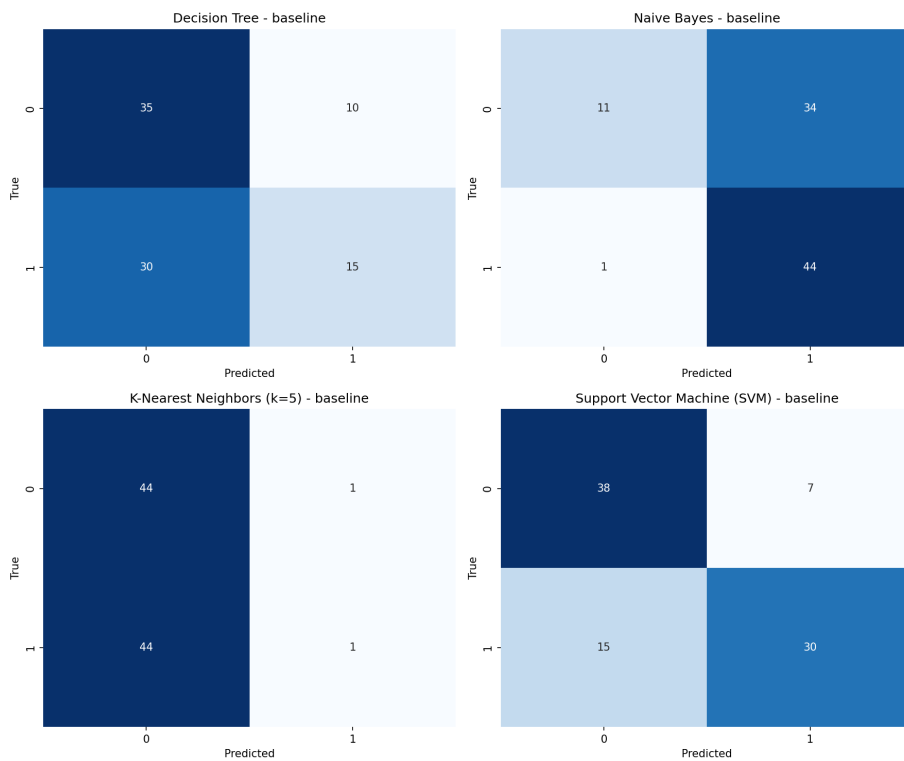


Figure 2: Matrices de confusión obtenidas para cada uno de los modelos entrenados.

El entrenamiento se realizó con las funciones `DecisionTreeClassifier`, `GaussianNB`, `KNeighborsClassifier` (con  $k = 5$ ) y `SVC` de la biblioteca Scikit-learn. Previo al entrenamiento, el conjunto de datos se normalizó utilizando el objeto `StandardScaler` de la misma biblioteca. Este método de normalización aplica la transformación  $z = (x - u)/s$ , donde  $u$  corresponde al promedio y  $s$  a la desviación estándar del conjunto.

Cada uno de estos modelos fue evaluado utilizando validación cruzada estratificada de 5 particiones, seleccionada para obtener estimaciones de desempeño más estables dada la cantidad relativamente pequeña de datos disponibles. Para comparar el desempeño de los modelos se utilizaron las métricas de accuracy, precision, recall y F1-score. Los resultados se resumen en la Figura 1, mientras que las matrices de confusión correspondientes se muestran en la Figura 2.

La métrica accuracy representa la proporción total de clasificaciones correctas realizadas por el modelo. Precision mide qué proporción de las predicciones positivas realmente corresponde a imágenes contaminadas, mientras que recall evalúa qué porcentaje de las imágenes contaminadas presentes en el conjunto fue detectado correctamente. Finalmente, el F1-score corresponde a la media armónica entre precision y recall, por lo que proporciona una medida balanceada entre ambas métricas.

Además de las métricas numéricas, se utilizaron matrices de confusión para analizar con mayor detalle el comportamiento de cada clasificador. Estas matrices muestran la cantidad de verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos generados por el modelo, permitiendo identificar qué tipos de errores ocurren con mayor frecuencia y evaluar la capacidad del clasificador para distinguir correctamente entre imágenes contaminadas y no contaminadas.

Como muestran las métricas evaluadas, el modelo SVM superó consistentemente a los demás modelos en todas las métricas por un margen considerable. Las matrices de confusión también muestran una clara ventaja del modelo SVM, ya que es capaz de clasificar la mayoría de imágenes correctamente y presenta un sesgo bajo, es decir, clasifica ejemplos positivos y negativos correctamente con una frecuencia similar.

El desempeño superior del modelo SVM puede atribuirse a su capacidad para trabajar eficientemente en espacios de alta dimensionalidad [1], característica particularmente relevante en este problema debido a que cada píxel de la imagen constituye una característica de entrada. Además, el uso de kernels permite modelar fronteras de decisión complejas entre clases contaminadas y no contaminadas. En contraste, modelos como KNN pueden verse afectados negativamente por la alta dimensionalidad [2]. Asimismo, los árboles de decisión presentan una mayor tendencia al sobreajuste en conjuntos de datos pequeños y ruidosos [3].

El análisis de las matrices de confusión también permite observar que los falsos negativos representan uno de los errores más críticos del sistema, ya que implican clasificar una imagen contaminada como no contaminada. En un entorno industrial real, este tipo de error podría permitir el paso de productos defectuosos a etapas posteriores de producción o incluso al consumidor final. Por esta razón, métricas como recall adquieren una importancia particular en

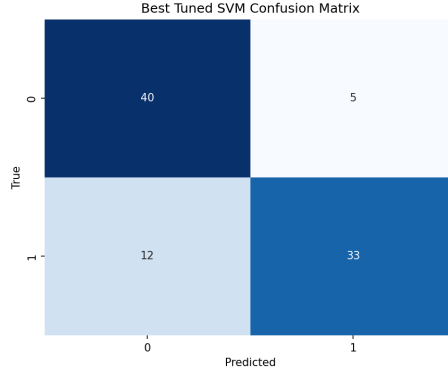


Figure 3: Matriz de confusión del algoritmo SVM.

este tipo de aplicaciones.

Debido a su desempeño superior, el modelo SVM fue seleccionado para un ajuste adicional de hiperparámetros. Para este modelo, el proceso consistió en variar el parámetro de regularización  $C$  y evaluar diferentes funciones kernel con el fin de determinar la configuración que produjera el mejor desempeño.

Para la configuración óptima del modelo SVM según “accuracy” las métricas finales se presentan en la Tabla 1 y la Figura 3. Este modelo se adjunta en el archivo `C22706_Guillermo_Escobar.joblib`.

Table 1: Métricas del algoritmo SVM.

| Modelo                       | Accuracy | Precision | Recall | F1-score |
|------------------------------|----------|-----------|--------|----------|
| Support Vector Machine (SVM) | 0.8111   | 0.8188    | 0.8111 | 0.8100   |

### 3.1 Limitaciones del modelo

A pesar de los resultados obtenidos, el sistema presenta varias limitaciones importantes. En primer lugar, el tamaño del conjunto de datos utilizado es relativamente pequeño, lo que limita la capacidad de generalización de los modelos entrenados. Además, la conversión de las imágenes a formato binario elimina información de intensidad y textura que podría ser útil para la clasificación.

Asimismo, las imágenes utilizadas fueron obtenidas en un ambiente simulado y relativamente controlado. En un entorno industrial real podrían existir variaciones importantes de iluminación, orientación de objetos, ruido visual y tipos de contaminación no presentes en el conjunto de entrenamiento. Estas condiciones podrían reducir significativamente el desempeño observado durante las pruebas.

## 4 Conclusiones

Se entrenaron y evaluaron distintos modelos clásicos de aprendizaje automático para la clasificación de imágenes contaminadas en una línea de producción simulada. Entre los modelos evaluados, SVM obtuvo consistentemente el mejor desempeño en todas las métricas consideradas, mostrando además una matriz de confusión balanceada y un bajo sesgo entre clases.

Los resultados obtenidos muestran que incluso algoritmos clásicos pueden alcanzar desempeños adecuados en tareas de visión computacional relativamente simples cuando se realiza un preprocesamiento apropiado de los datos. Sin embargo, también se identificaron limitaciones importantes relacionadas con la cantidad de datos disponibles y la naturaleza simplificada del entorno de pruebas.

Como trabajo futuro, podría explorarse el uso de redes neuronales convolucionales (CNN), técnicas de aumento de datos y conjuntos de imágenes más grandes y variados para mejorar la robustez y capacidad de generalización del sistema.

## 5 Uso de IA

Se utilizó Github Copilot para escribir parte del código de preprocesamiento, el código de entrenamiento y la documentación final. El producto final fue revisado detalladamente.

## References

- [1] Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. Machine Learning, 20(3), 273–297.
- [2] Bellman, R. (1961). *Adaptive Control Processes: A Guided Tour*. Princeton University Press.
- [3] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.